

Análise do Impacto do Carregamento Paralelo de Dados no Treinamento de Redes Neurais Convolucionais

Antonio Ambrosio da Silva

IFMG - Campus Bambuí

Faz. Varginha - Rodovia Bambuí/Medeiros - Bambuí, MG

antonioambrosiooficial@gmail.com

Euler Gomes da Rocha

IFMG - Campus Bambuí

Faz. Varginha - Rodovia Bambuí/Medeiros - Bambuí, MG

eulergomesrocha@gmail.com

Vinícius Miguel Leite Gomes de Souza

IFMG - Campus Bambuí

Faz. Varginha - Rodovia Bambuí/Medeiros - Bambuí, MG

viniciusmiguelldgs@gmail.com

RESUMO

Este trabalho analisa como o carregamento paralelo de dados afeta o treinamento de redes neurais para classificação de imagens médicas. Avalia o impacto de `num_workers` e `batch_size` em tempo, throughput e acurácia, usando o dataset HAM10000 em um desktop e no Google Colab. Os resultados mostram que o paralelismo pode reduzir significativamente o tempo em até mais de 5 vezes, mas há saturação devido a limitações de hardware. O `batch_size` teve impacto menor. Conclui-se que a escolha adequada desses parâmetros e do hardware é essencial para otimizar o treinamento.

PALAVRAS CHAVE. Redes Neurais Convolucionais, Paralelismo, Carregamento de Dados, Imagens Médicas

ABSTRACT

This work analyzes how parallel data loading affects the training of neural networks for medical image classification. It evaluates the impact of `num_workers` and `batch_size` on time, throughput, and accuracy, using the HAM10000 dataset on a desktop and on Google Colab. The results show that parallelism can significantly reduce training time by more than 5 times, but there is saturation due to hardware limitations. The `batch_size` had a smaller impact. It is concluded that the proper choice of these parameters and the hardware is essential to optimize training.

KEYWORDS. Convolutional Neural Networks, Parallelism, Data Loading, Medical Images.

1. Introdução

O avanço recente das técnicas de aprendizado de máquina, em especial das redes neurais profundas, tem impulsionado aplicações relevantes na área da saúde, como a classificação automática de imagens médicas. Modelos baseados em redes neurais convolucionais (CNNs) têm apresentado alto desempenho em tarefas como detecção de doenças em exames de imagem, contribuindo para apoio ao diagnóstico clínico LeCun et al. [2015]; Krizhevsky et al. [2012].

Entretanto, o treinamento desses modelos envolve grande volume de dados e elevado custo computacional, tornando-se um desafio relevante em termos de desempenho e eficiência. Uma parte significativa desse custo está associada ao carregamento e pré-processamento dos dados, que pode se tornar um gargalo no pipeline de treinamento Goodfellow et al. [2016].

Nesse contexto, técnicas de paralelismo têm sido amplamente exploradas para reduzir o tempo de treinamento. Em particular, o uso de múltiplos processos para carregamento de dados, como implementado no parâmetro *num_workers* do DataLoader em bibliotecas como o *PyTorch*, permite que operações de leitura e transformação de dados sejam realizadas de forma concorrente, potencialmente aumentando o *throughput* do sistema.

A análise do impacto do paralelismo pode ser fundamentada pela Lei de Amdahl, que estabelece limites teóricos para o ganho de desempenho em sistemas paralelos, considerando a fração do código que permanece sequencial Amdahl [1967]. De acordo com essa lei, o ganho obtido com paralelização é limitado pela parte não paralelizável do processo, o que torna essencial a análise experimental do comportamento do sistema.

Dessa forma, este trabalho tem como objetivo investigar como o carregamento paralelo de dados influencia o desempenho do treinamento de redes neurais convolucionais aplicadas à classificação de imagens médicas. Para isso, são conduzidos experimentos variando o número de processos de carregamento (*num_workers*) e o tamanho do lote (*batch_size*), avaliando métricas como tempo de treinamento, *throughput* e acurácia do modelo.

Como principal contribuição, este trabalho apresenta uma análise experimental detalhada do impacto do paralelismo no carregamento de dados em diferentes ambientes computacionais, evidenciando limites práticos e diretrizes para otimização do treinamento de redes neurais.

2. Trabalhos Relacionados

O desempenho do treinamento de redes neurais profundas tem sido amplamente estudado, especialmente no contexto de grandes volumes de dados e alto custo computacional. Estudos recentes mostram que, além do modelo em si, o pipeline de dados exerce papel fundamental no tempo total de treinamento, podendo se tornar um gargalo significativo Mohan et al. [2020].

Nesse sentido, diversos trabalhos investigam o impacto do carregamento de dados no desempenho de modelos de aprendizado profundo. Svogor et al. [2022] analisam o funcionamento do DataLoader do PyTorch em ambientes com alta latência de armazenamento, demonstrando que otimizações no carregamento podem reduzir significativamente o tempo de treinamento, alcançando melhorias de até 12 vezes em determinados cenários. De forma semelhante, Sun et al. [2022] propõem um novo mecanismo de carregamento de dados para treinamento distribuído, obtendo ganhos expressivos de desempenho ao otimizar o acesso e o paralelismo no sistema de I/O.

Além disso, Mohan et al. [2020] demonstram que o tempo de treinamento de redes neurais pode ser fortemente impactado por períodos de espera por dados, conhecidos como *data stalls*, evidenciando a importância de otimizar o pipeline de entrada. Os autores mostram que, em alguns casos, o tempo de espera por dados pode dominar o tempo total de execução.

No contexto de frameworks de aprendizado profundo, estudos como o de Aach et al. [2023] analisam o desempenho de diferentes abordagens de treinamento distribuído, destacando a importância da paralelização para lidar com grandes volumes de dados e melhorar a escalabilidade dos modelos.

Especificamente no PyTorch, o parâmetro *num_workers* do *DataLoader* permite o carregamento paralelo de dados por meio de múltiplos processos. O uso adequado desse parâmetro pode aumentar o throughput do sistema ao permitir que o carregamento e o processamento de dados ocorram simultaneamente com o treinamento do modelo Svogor et al. [2022]. No entanto, o aumento do número de processos também pode introduzir overhead e competição por recursos, o que pode limitar os ganhos de desempenho.

Dessa forma, a literatura evidencia que o carregamento de dados é um componente crítico no treinamento de redes neurais profundas, sendo necessário encontrar um equilíbrio entre paralelismo e custo computacional. Este trabalho se insere nesse contexto, investigando experimentalmente o impacto do parâmetro *num_workers* e do tamanho do lote no desempenho do treinamento de modelos de classificação de imagens médicas.

3. Metodologia

A metodologia adotada neste trabalho tem como objetivo avaliar o impacto do paralelismo no carregamento de dados durante o treinamento de redes neurais convolucionais. Para isso, foram conduzidos experimentos controlados variando o número de workers do *DataLoader* e o tamanho do *batch*, analisando seu efeito no desempenho do treinamento.

3.1. Preparação dos Dados

Foi utilizado o dataset HAM10000, composto por aproximadamente 10.000 imagens de lesões de pele distribuídas em sete classes. As imagens foram organizadas em diretórios compatíveis com a estrutura esperada pela classe *ImageFolder* da biblioteca PyTorch.

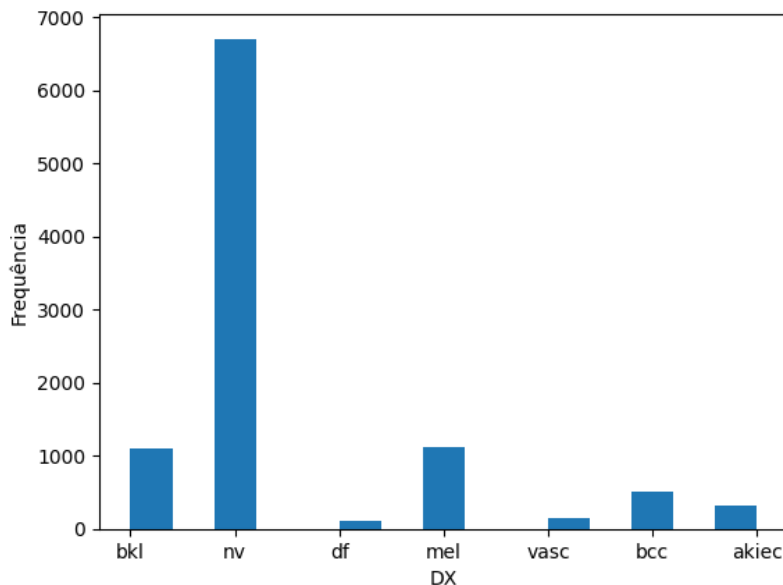


Figura 1: Histograma do dataset
Fonte: Autores (2026)

A figura 1 representa o histograma do *dataset*, mostrando a recorrência das sete classes presentes. Nota-se que há um desbalanceamento entre as classes presentes, onde a classe 'nv' apresenta uma recorrência significativamente maior do que as outras classes.

As imagens passaram por um processo de pré-processamento utilizando `torchvision.transforms`, incluindo redimensionamento para 128×128 pixels, conversão para tensor e normalização dos valores de pixel. Essas transformações garantem compatibilidade com o modelo e contribuem para a estabilidade do treinamento.

O conjunto de dados foi dividido em subconjuntos de treino e validação, permitindo a avaliação do desempenho do modelo durante o processo de aprendizado.

3.2. Implementação da Função de Treinamento

Foi implementada uma função de treinamento parametrizada que recebe como entrada o número de workers (`num_workers`) e o tamanho do lote (`batch_size`). Essa função é responsável por:

- Criar os *DataLoaders* de treino e validação com os parâmetros especificados;
- Inicializar o modelo de rede neural convolucional;
- Definir o otimizador Adam com taxa de aprendizado de 0.001;
- Utilizar a função de perda *CrossEntropyLoss*;
- Executar o treinamento por 5 épocas;
- Medir métricas de desempenho, incluindo tempo por época, tempo total, throughput (imagens por segundo) e acurácia no conjunto de validação.

O modelo utilizado consiste em uma rede neural convolucional com três camadas convolucionais, seguidas por camadas de *pooling*, camadas totalmente conectadas e uma camada de *dropout* para reduzir *overfitting*.

3.3. Experimentos

Os experimentos foram organizados em duas etapas principais, com o objetivo de analisar separadamente o impacto do paralelismo no carregamento de dados e a influência do tamanho do lote no desempenho do treinamento.

3.3.1. Variação do Número de Workers

Nesta etapa, o tamanho do *batch* foi mantido fixo em 64, enquanto o número de workers do *DataLoader* foi variado entre os valores de 0, 2, 4, 6 e 8.

Para cada configuração, foram coletadas métricas como tempo por época, tempo total, throughput e acurácia. Cada experimento foi realizado com um modelo inicializado com pesos aleatórios, garantindo independência entre os testes. O objetivo foi avaliar como o paralelismo no carregamento de dados influencia o desempenho do treinamento.

3.3.2. Variação do Batch Size

Na segunda etapa, o número de workers foi fixado em um valor intermediário (tipicamente 4), enquanto o tamanho do *batch* foi variado entre os valores de 32, 64, 128 e 256, quando permitido pela memória disponível.

Para os testes variando o tamanho do *batch*, foi seguido o mesmo padrão da etapa anterior, coletando as mesmas métricas e gerando um novo modelo por valor de *batch*. Essa análise permitiu investigar a relação entre o tamanho do lote e o desempenho computacional, bem como sua interação com o paralelismo no carregamento de dados.

3.4. Materiais e tecnologias

A implementação foi realizada na linguagem de programação Python (versão 3.11) Python Software Foundation [2026] em conjunto com o ambiente interativo Jupyter Notebook Project Jupyter [2026]. Para a construção do modelo foi utilizado a biblioteca de código aberto Pytorch Contributors [2026]

Para melhores testes e verificação do funcionamento do projeto, foram utilizados dois diferentes *hardwares*, sendo eles:

- Um computador *Desktop* com um AMD Ryzen 5 7600X, 32GB de memória RAM e uma placa de vídeo NVIDIA RTX 5070, que conta com 6.144 núcleos CUDA.
- A ferramenta gratuita em nuvem Google Colab, que conta com uma NVIDIA Tesla T4, que conta com 2.560 núcleos CUDA.

Ambos os dois diferentes *hardwares* foram testados utilizando o mesmo código-fonte, para garantir a equiparidade dos resultados.

4. Resultados e Discussão

Nesta seção, são apresentados e analisados os resultados obtidos para cada configuração de hardware descrita na Seção 3.4, com foco no impacto dos parâmetros *num_workers* e *batch_size* no desempenho e na acurácia do modelo.

4.1. Hardware 1

Utilizando o Hardware 1 (desktop equipado com processador AMD Ryzen 5 7600X, 32 GB de memória RAM e GPU NVIDIA RTX 5070), observou-se um impacto significativo do paralelismo no carregamento de dados sobre o desempenho do treinamento.

num_workers	Tempo/época (s)	Tempo total(s)	Throughput (img/s)	Acurácia (%)
0	21.66	110.13	316,95	74,90
2	11.36	56.79	614.59	74,13
4	6.24	31.18	1119.44	74,90
6	4.55	22.89	1524.91	74,59
8	4.09	20.73	1683.77	73,71

Tabela 1: Variação de *num_workers* (batch_size=64)

Conforme apresentado na Tabela 1, o aumento do parâmetro *num_workers* de 0 para 8 resultou em uma redução expressiva no tempo por época, passando de 21,66 s para 4,09 s, o que corresponde a um ganho superior a 80%. De forma consistente, o *throughput* aumentou de 316,95 para 1683,77 imagens por segundo, evidenciando a mitigação do gargalo de entrada/saída (I/O) por meio do carregamento paralelo de dados.

Entretanto, observa-se um comportamento de saturação nos ganhos de desempenho. A partir de 6 *workers*, a redução no tempo por época torna-se marginal, indicando que outros componentes do sistema passam a limitar o desempenho global, como CPU, largura de banda de memória e comunicação com a GPU. Esse comportamento está em conformidade com a Lei de Amdahl, que estabelece limites para o ganho de desempenho em sistemas paralelos.

Em relação à acurácia, os resultados permaneceram estáveis, variando entre 73% e 75%, indicando que o aumento do paralelismo no carregamento de dados não impacta significativamente a qualidade do modelo, mas apenas o tempo de treinamento.

batch_size	Tempo/época (s)	Throughput (img/s)	Acurácia (%)
32	5.84	1175.18	74.72
64	6.20	1136.05	73.81
128	6.09	1141.43	74.82
256	6.20	1120.25	72.53

Tabela 2: Variação de batch_size (num_workers=4)

No que se refere ao tamanho do lote (Tabela 2), observa-se que a variação do *batch_size* não resultou em mudanças expressivas no tempo por época nem no *throughput*, cujos valores permaneceram próximos em todas as configurações avaliadas.

Por outro lado, a acurácia apresentou pequenas variações, com melhor resultado obtido para *batch_size* 128 (74,82%), sugerindo um possível equilíbrio entre estabilidade do gradiente e eficiência computacional. Valores muito elevados de *batch_size*, como 256, apresentaram leve queda de desempenho em termos de generalização.

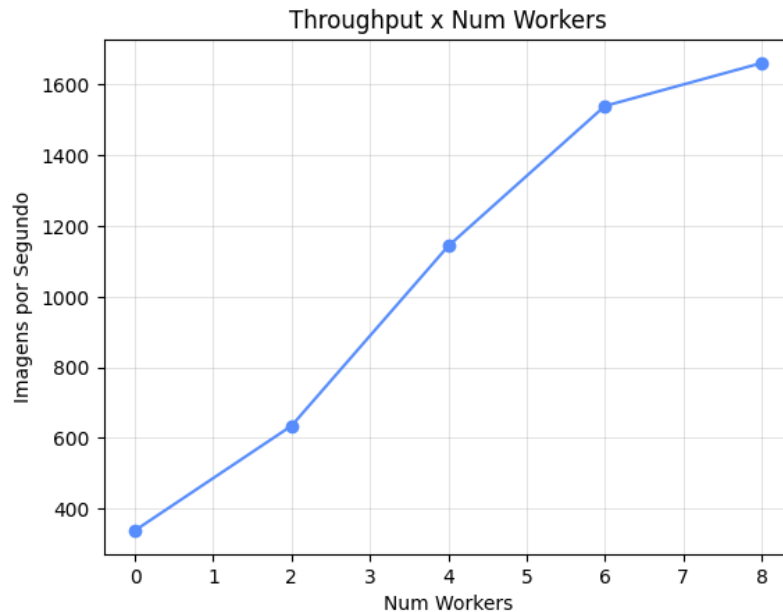


Figura 2: Evolução do *throughput* em função do número de *workers*, evidenciando ponto de saturação do paralelismo

Fonte: Autores (2026)

A Figura 2 ilustra o comportamento do *throughput* em função do número de *workers*, evidenciando um crescimento acentuado até 6 *workers*, seguido de estabilização. Esse resultado reforça a presença de um ponto de saturação do sistema.

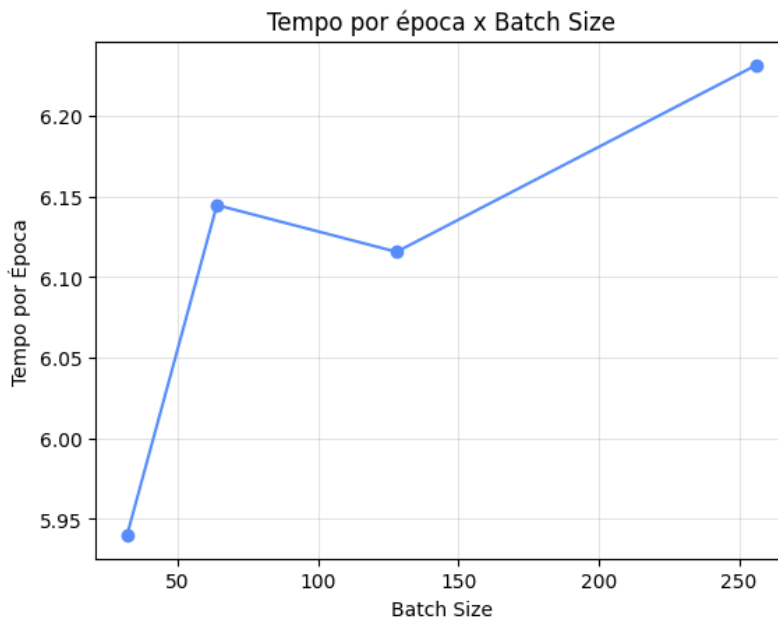


Figura 3: Tempo por época em função do *batch_size*, evidenciando baixa sensibilidade do desempenho
Fonte: Autores (2026)

A Figura 3 apresenta o tempo por época em função do *batch_size*, indicando variações pouco significativas entre as configurações, com leve vantagem para o valor 128.

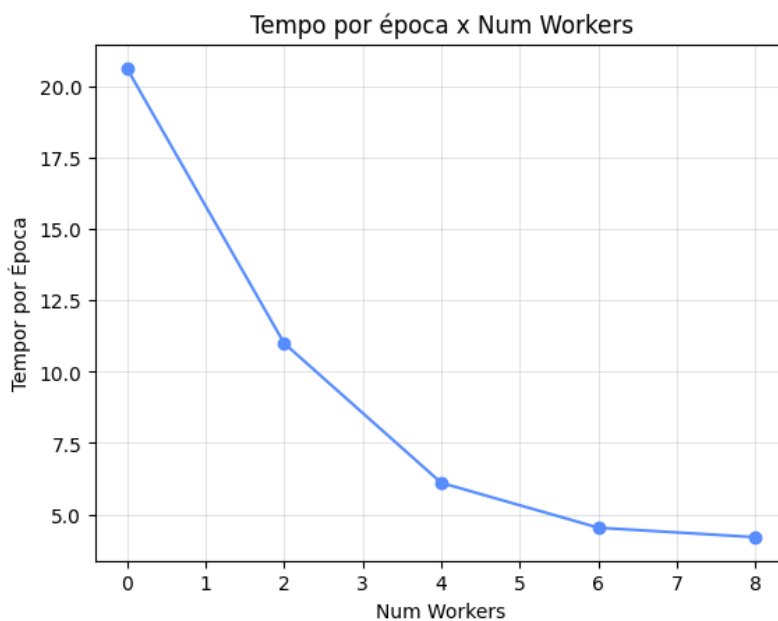


Figura 4: Tempo por época em função do número de *workers*, evidenciando redução inicial seguida de saturação
Fonte: Autores (2026)

Por fim, a Figura 4 demonstra a redução do tempo por época com o aumento de *num_workers*, sendo o melhor resultado obtido com 8 *workers*, ainda que com ganhos marginais em relação a 6.

Esse comportamento experimental está diretamente alinhado com a Lei de Amdahl, evidenciando que, à medida que a fração paralelizável do sistema é explorada, o ganho adicional de desempenho torna-se limitado pela porção sequencial do pipeline.

De forma geral, os resultados indicam que, para o hardware analisado, o parâmetro *num_workers* exerce influência significativamente maior sobre o desempenho do que o *batch_size*, sendo um fator crítico para a otimização do tempo de treinamento.

4.2. Hardware 2

Utilizando o Hardware 2 (ambiente Google Colab com GPU NVIDIA Tesla T4), os resultados apresentaram comportamento distinto em relação ao Hardware 1, especialmente no que se refere ao paralelismo no carregamento de dados.

num_workers	Tempo/época (s)	Tempo total (s)	Throughput (img/s)	Acurácia (%)
0	53,15	265,79	131,33	74,59
2	43,12	215,58	161,91	75,26
4	43,57	217,86	160,22	74,17
6	44,02	220,12	158,57	73,81
8	45,56	227,79	153,24	73,74

Tabela 3: Variação de *num_workers* (batch_size=64)

De acordo com a Tabela 3, o aumento de *num_workers* de 0 para 2 proporcionou melhora significativa no desempenho, com redução do tempo por época de 53,15 s para 43,12 s e aumento do *throughput* de 131,33 para 161,91 imagens por segundo.

Entretanto, a partir desse ponto, o aumento adicional do número de *workers* não resultou em ganhos, sendo observado inclusive um leve aumento no tempo de execução para valores superiores a 4 *workers*. Esse comportamento indica que o sistema atinge rapidamente um ponto de saturação, no qual o custo de gerenciamento de múltiplos processos e a contenção de recursos superam os benefícios do paralelismo.

Esse efeito pode estar associado às limitações do ambiente do Google Colab, como restrições de CPU, menor controle sobre recursos de hardware, contenção em memória compartilhada e possíveis limitações de I/O.

Em relação à acurácia, os valores permaneceram estáveis entre 73% e 75%, com melhor desempenho observado para 2 *workers*, reforçando que o paralelismo impacta principalmente o tempo de execução, sem afetar significativamente a qualidade do modelo.

batch_size	Tempo/época (s)	Throughput (img/s)	Acurácia (%)
32	44,10	158,30	75,03
64	43,46	160,62	74,72
128	43,34	161,08	74,34
256	44,66	156,31	71,75

Tabela 4: Variação de *batch_size* (num_workers=4)

Na análise do *batch_size* (Tabela 4), o tempo por época e o *throughput* permaneceram praticamente constantes entre as configurações, com pequenas variações. O maior *throughput* foi obtido com *batch_size* 128, enquanto a maior acurácia foi observada com *batch_size* 32 (75,03%).

Por outro lado, o aumento do *batch_size* para 256 resultou em queda significativa da acurácia (71,75%), o que pode estar relacionado à redução da frequência de atualização dos pesos e à possível degradação da capacidade de generalização do modelo.

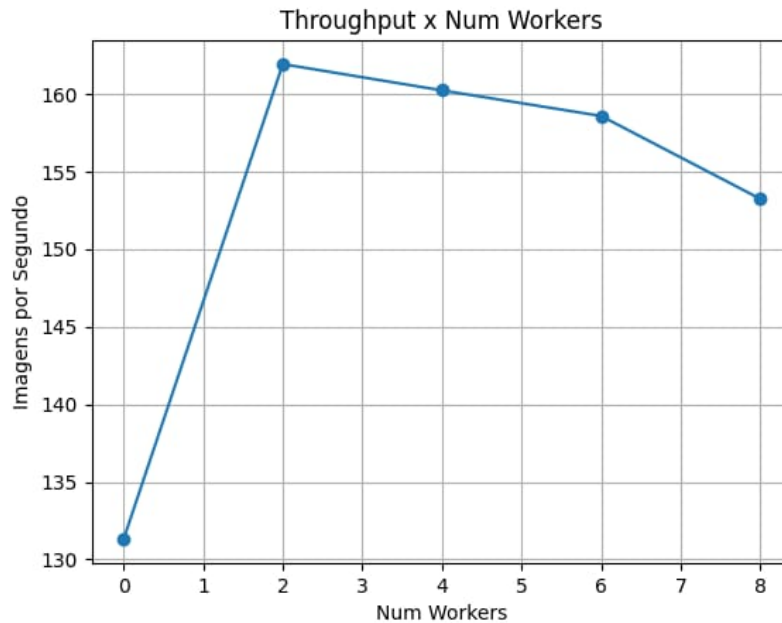


Figura 5: Evolução do *throughput* em função do número de *workers*, evidenciando ponto de saturação do paralelismo

Fonte: Autores (2026)

A Figura 5 apresenta o comportamento do *throughput* em função do número de *workers*. Observa-se um crescimento significativo no *throughput* ao aumentar de 0 para 2 *workers*, passando de aproximadamente 131 para 162 imagens por segundo, o que reforça a hipótese de eliminação de gargalos de I/O. Após esse ponto, o *throughput* passa a apresentar uma tendência de queda gradual, indicando que o aumento do paralelismo não se traduz em maior eficiência.

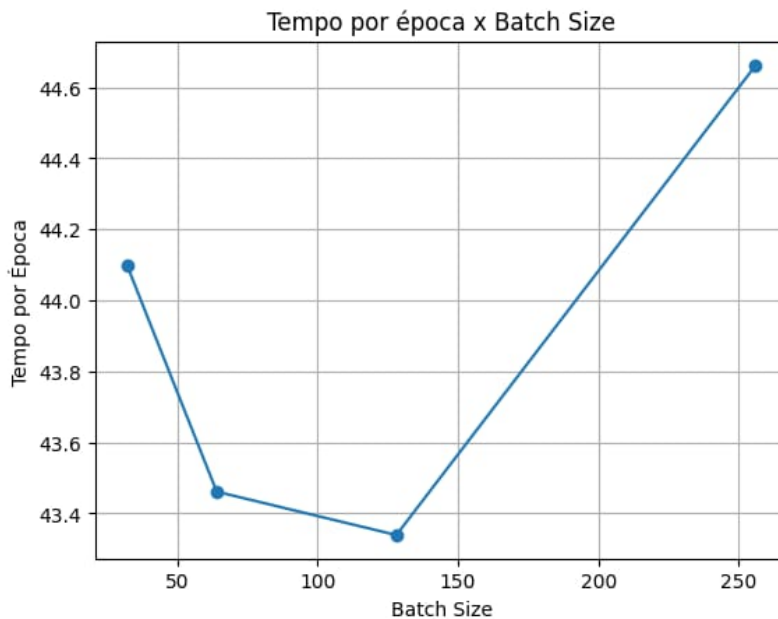


Figura 6: Tempo por época em função do *batch_size*, evidenciando baixa sensibilidade do desempenho
Fonte: Autores (2026)

A Figura 6 apresenta o impacto do tamanho do lote (*batch_size*) no tempo por época. Os resultados mostram que o tempo por época apresenta baixa sensibilidade à variação do tamanho do lote, mantendo-se relativamente estável entre os valores analisados. Observa-se uma leve redução até o *batch_size* 128, onde é atingido o menor tempo médio.

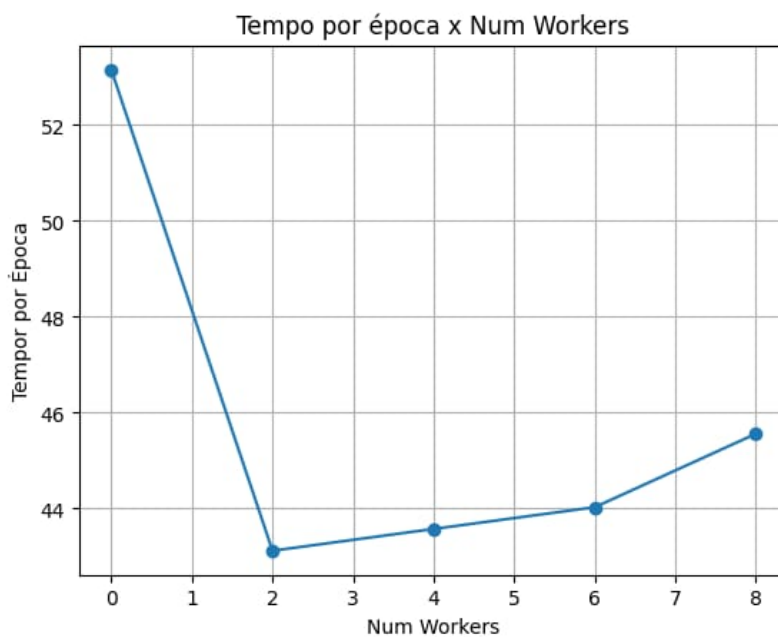


Figura 7: Tempo por época em função do número de *workers*, evidenciando redução inicial seguida de saturação
Fonte: Autores (2026)

A Figura 7 apresenta a variação do tempo por época em função do número de *workers*. Observa-se uma redução acentuada no tempo de execução ao passar de 0 para 2 *workers*, indicando que o paralelismo inicial no carregamento de dados elimina um gargalo significativo de entrada/saída. Entretanto, a partir de 2 *workers*, verifica-se uma inversão de tendência, com aumento gradual do tempo por época conforme o número de *workers* cresce.

De forma geral, os resultados indicam que, em ambientes com recursos mais limitados ou compartilhados, como o Google Colab, o uso excessivo de paralelismo pode ser contraproducente, sendo recomendada a utilização de um número moderado de *workers* para obtenção do melhor desempenho.

4.3. Comparação entre os Hardwares

A comparação entre os dois ambientes experimentais evidencia diferenças significativas no comportamento do paralelismo. No Hardware 1, observou-se escalabilidade mais eficiente com o aumento do número de *workers*, com ganhos expressivos de desempenho até a região de saturação.

Por outro lado, no ambiente do Google Colab (Hardware 2), o ponto de saturação foi atingido precocemente, com melhor desempenho obtido com apenas 2 *workers*. O aumento adicional do paralelismo resultou em degradação de desempenho, indicando maior sensibilidade à contenção de recursos.

Esses resultados demonstram que a eficiência do paralelismo está diretamente relacionada às características do hardware, como número de núcleos de CPU, largura de banda de memória e desempenho do subsistema de I/O. Dessa forma, a escolha do valor de *num_workers* deve ser realizada de forma empírica e dependente do ambiente de execução.

5. Conclusão

Este trabalho investigou o impacto do carregamento paralelo de dados no desempenho do treinamento de redes neurais convolucionais aplicadas à classificação de imagens médicas. Através de experimentos controlados, foi possível analisar a influência dos parâmetros *num_workers* e *batch_size* em diferentes configurações de hardware.

Os resultados demonstraram que o uso de múltiplos *workers* no *DataLoader* pode reduzir significativamente o tempo de treinamento, aumentando o *throughput* do sistema de forma expressiva. Em particular, no hardware de maior desempenho, observou-se ganhos superiores a 5 vezes quando comparado à execução sequencial. No entanto, também foi identificado um ponto de saturação, no qual o aumento adicional de *workers* resulta em ganhos marginais, evidenciando limitações impostas por recursos não paralelizáveis, conforme previsto pela Lei de Amdahl.

Por outro lado, a variação do *batch_size* apresentou impacto menos significativo no desempenho computacional, embora pequenas variações na acurácia tenham sido observadas, sugerindo a existência de um compromisso entre eficiência e qualidade do treinamento.

Além disso, os experimentos realizados em diferentes hardwares evidenciam que o ganho obtido com paralelismo depende diretamente das características da máquina, como número de núcleos de CPU, capacidade de memória e desempenho de I/O.

Dessa forma, conclui-se que a otimização do pipeline de dados e o *hardware* utilizado são fatores essenciais para o treinamento eficiente de redes neurais profundas, sendo o ajuste adequado do parâmetro *num_workers* uma estratégia simples, porém altamente eficaz, para a melhoria do desempenho em pipelines de treinamento baseados em aprendizado profundo.

Como trabalhos futuros, sugere-se a investigação de técnicas mais avançadas de paralelismo, como pré-busca (*prefetching*), uso de memória compartilhada, armazenamento em SSD de alta velocidade e treinamento distribuído em múltiplas GPUs, bem como a avaliação em datasets maiores e modelos mais complexos.

6. Agradecimentos

Os autores agradecem ao Instituto Federal de Educação, Ciência e Tecnologia de Minas Gerais (IFMG) pelo apoio institucional, bem como à Fundação de Amparo à Pesquisa do Estado de Minas Gerais (FAPEMIG), à Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) e ao Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) pelo apoio ao desenvolvimento da pesquisa científica no Brasil.

Disponibilidade de Dados e Código

Para fins de transparência e reprodutibilidade científica, o código-fonte da rede, os *scripts* de treinamento e o conjunto de dados estão disponíveis publicamente no repositório GitHub em: https://github.com/eulergomees/parallel_loading.

O repositório está estruturado com diretórios de suporte, arquivos de requisitos (*requirements.txt*), modelos pré-treinados e um *Jupyter Notebook* detalhando todas as etapas da implementação em Python e PyTorch.

Referências

- Aach, M. et al. (2023). Large scale performance analysis of distributed deep learning frameworks. *Journal of Big Data*.
- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Conference Proceedings*.
- Contributors, P. (2026). Pytorch. URL <https://pytorch.org/>. 2026-03-31.
- Goodfellow, I., Bengio, Y., e Courville, A. (2016). *Deep Learning*. MIT Press.
- Krizhevsky, A., Sutskever, I., e Hinton, G. (2012). Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*.
- LeCun, Y., Bengio, Y., e Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444.
- Mohan, J. et al. (2020). Analyzing and mitigating data stalls in dnn training. *arXiv preprint arXiv:2007.06775*.
- Project Jupyter (2026). Project Jupyter | Home. URL <https://jupyter.org/>.
- Python Software Foundation (2026). Welcome to Python.org. URL <https://www.python.org/>.
- Sun, B. et al. (2022). Solar: A highly optimized data loading framework. *arXiv preprint arXiv:2211.00224*.
- Svogor, I. et al. (2022). Profiling and improving the pytorch dataloader for high-latency storage. *arXiv preprint arXiv:2211.04908*.